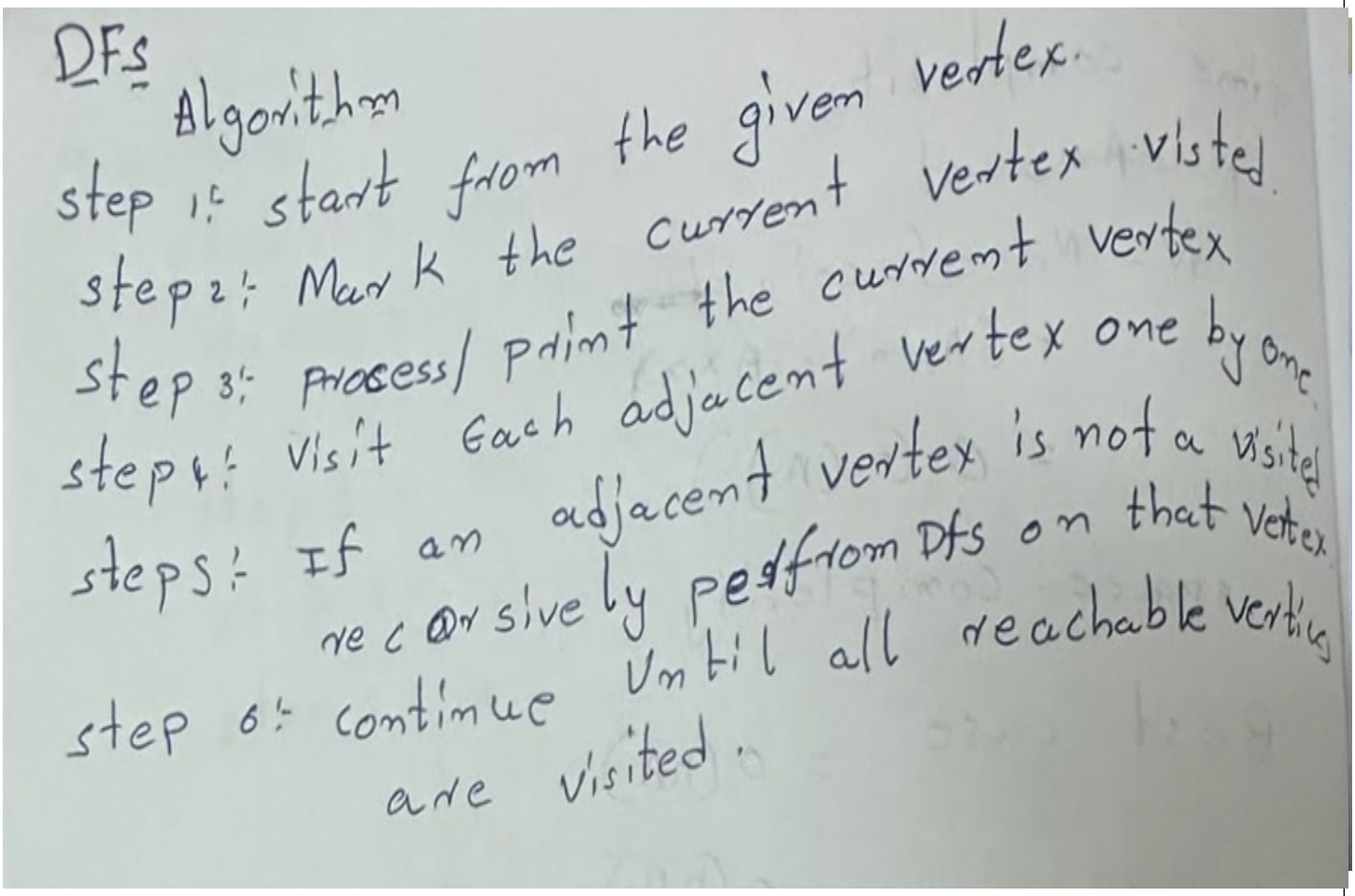
Practical: 8	Implementation of Graph and Searching (DFS and BFS).
22/09/2026	

Aim: To implement of Graph and Searching (DFS and BFS).

Algorithm:



BFS

Algo.

step1 :- Create An Empty Queue.

step2 :- Mark the starting vertex as visted.

step3 :- Insert the starting vertex into Queue

step4 :- while the Queue is not empty.

- Remove the front vertex from the Queue
- Process /Print it.
- visited all its unvisited adjacent vertices.
- Mark each adjacent vertex as visited
- Insert them into the Queue

step 5 :- continue until the Queue becomes empty.

Program:

```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
   // Static adjacency matrix
   int graph[5][5] =
   {
      {0, 1, 1, 0, 0},
      {1, 0, 0, 1, 1},
      {1, 0, 0, 0, 0},
      {0, 1, 0, 0, 0},
      {0, 1, 0, 0, 0}
   };

   int visited[5] = {0};

   // ------ DFS ----------
   cout << "DFS Traversal: ";

   int stack[5];
   int top = -1;

   stack[++top] = 0;
   visited[0] = 1;

   while (top >= 0)
   {
      int vertex = stack[top--];
      cout << vertex << " ";

      // Add adjacent vertices
      for (int i = 4; i >= 0; i--)
      {
         if (graph[vertex][i] == 1 && visited[i] == 0)
         {
            visited[i] = 1;
            stack[++top] = i;
         }
      }
   }

   // Reset visited array
```

```
    for (int i = 0; i < 5; i++)
        visited[i] = 0;

    // - - - - - - BFS - - - - - -
    cout << "\nBFS Traversal: ";

    queue<int> q;

    q.push(0);
    visited[0] = 1;

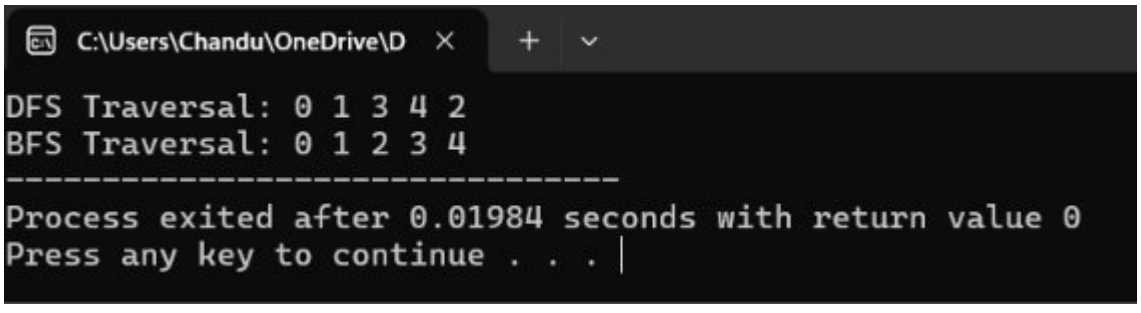
    while (!q.empty())
    {
        int vertex = q.front();
        q.pop();

        cout << vertex << " ";

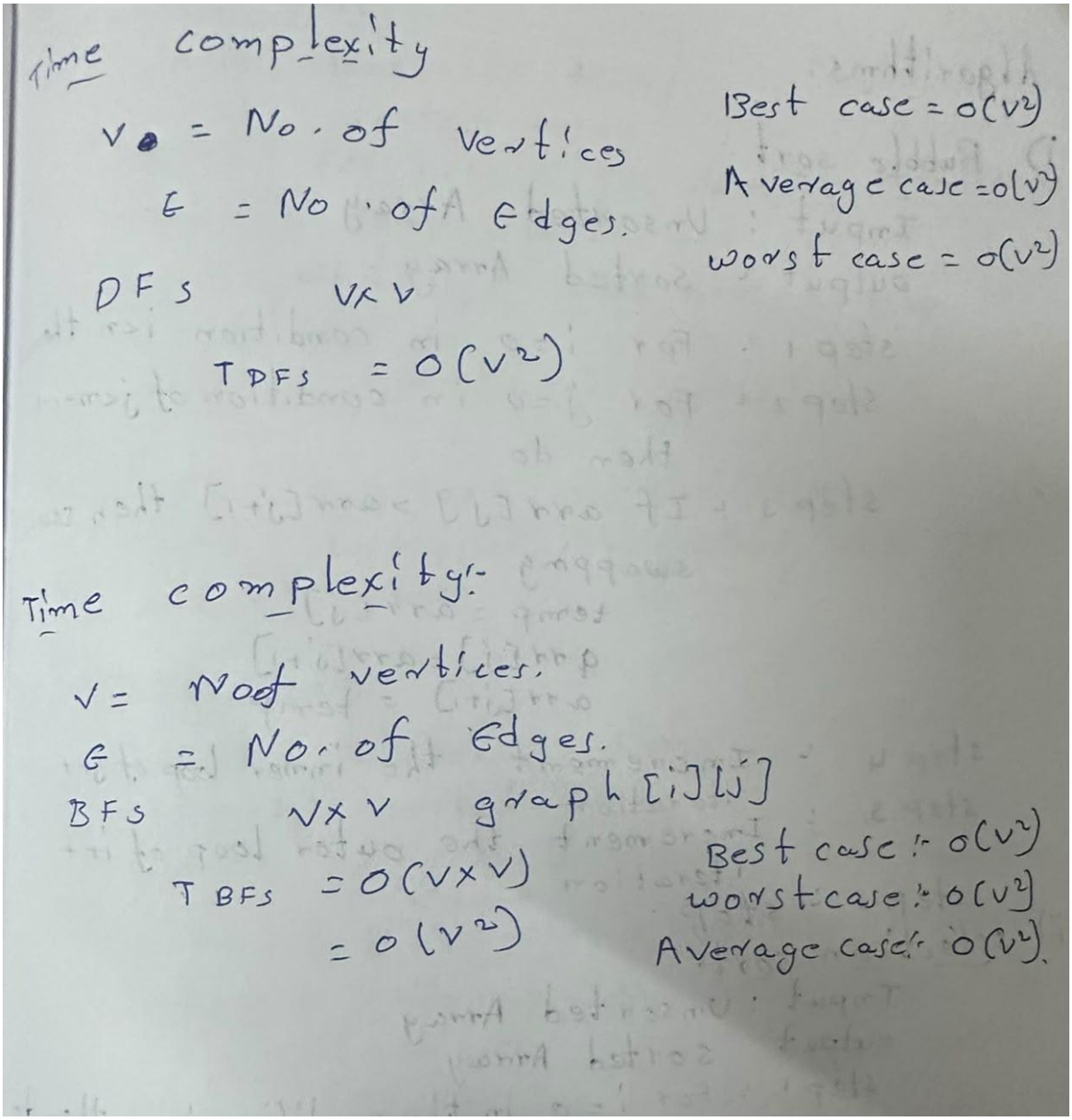
        for (int i = 0; i < 5; i++)
        {
            if (graph[vertex][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                q.push(i);
            }
        }
    }

    return 0;
}
```

Output:



Time Complexity:



Conclusion:

The Implementation of Graph and Searching DFS and BFS are successfully executed with the time complexity of $O(V^2)$
And same for all the three cases of best, average and worst.